

Archivara Agent: A Matched-Control No-Go for Cellular-Automata Search on Hadamard Order 668

Archivara research pipeline via Codex CLI

March 18, 2026

Problem. Evaluate whether a proposed “cellar automata” route can materially advance the search for a Hadamard matrix of order 668.

Validated scope. The executed work consists of two Hadamard-specific cellular-automata branches tied to the exact length-167, weight-80 cyclic obstruction of Constantine and Constantine [3] and the published mod-64 order-668 seed of Eliahou [5], both benchmarked against matched same-representation non-CA controls. The literal “cellar” reading survives only as an unexecuted reserve automaton-theoretic pivot.

Manuscript type. Reproducible negative-result packet with exact artifact provenance, fairness proofs for the benchmarked update rules, and publication-quality figures generated directly from checked-in JSON artifacts.

Abstract

We study a proposed automata-based route to the Hadamard order-668 problem under a strict evidence gate: every claim must be tied either to an exact mathematical argument or to a reproducible experiment on checked-in artifacts. The initial prompt suggested that a previously untried “cellar automata” method might find a Hadamard matrix of order 668. In the executed repository state, that phrase was operationalized first as cellular automata and only later reinterpreted literally as a reserve pushdown-style automaton. The validated contribution of the repository is therefore narrower than the prompt: two cellular-automata search branches were implemented and tested under matched controls.

The first branch, H1, acts on fixed-weight binary supports on the cycle $\mathbb{Z}/167\mathbb{Z}$ and targets the exact unresolved length-167, weight-80 cyclic autocorrelation obstruction from Constantine and Constantine [3]. The second branch, H2, acts on structured (q, s) states derived from the published mod-64 order-668 seed of Eliahou [5]. In both branches the serious non-CA baseline `direct_greedy` shares the same representation, neighborhood, move cap, step budget, and verifier, so the only algorithmic difference is the move-selection policy. We prove the relevant invariants, exact delta formulas, connectivity statements, and benchmark fairness properties for both implemented branches.

The measured outcome is negative. On the solved 4×79 same-template control, `direct_greedy` attains a better best-distance mean than `parallel_gain_ca` (41.5 versus 48.375) and wins on 11/16 matched seeds on the primary metric. On the exact 167/80 target, neither method finds a witness, and `direct_greedy` beats `parallel_gain_ca` on all 24/24 matched seeds, with best-distance means 147.9 versus 188.5. The small $n = 9$ structured ladder for H2 is only a weak sanity check because random controls solve most starts, while the decisive real order-668 test consists of one degraded start in which `parallel_gain_ca` and `direct_greedy` tie exactly on the best achieved modulus, ℓ_1 defect, defect count, and maximum defect magnitude. The only surviving scientific claim is therefore a narrow no-go result: under the executed same-representation, same-budget evaluations, these cellular-automata rules do not outperform their matched non-CA controls for the exact artifacts tested here. We close by isolating the literal “cellar” automaton as a reserve hypothesis rather than a validated result.

1 Introduction

The existence of a Hadamard matrix of order 668 remains a difficult search target, and recent work has sharpened the relevant exact subproblems in two different directions. On the cyclic side, Constantine and Constantine [3] isolates an unresolved length-167, weight-80 support with a fully specified periodic autocorrelation target. On the modular side, Eliahou [5] constructs a 64-modular Hadamard matrix of order 668, supplying a concrete near-solution but not a genuine Hadamard certificate. Those exact objects are unusually useful for evaluating heuristics because they fix the representation and the verifier before search begins.

The repository studied in this paper began from a user request that explicitly named “cellar automata” as a potentially untried method. The project did not validate a literal cellar automaton. Instead, the executed run first interpreted the phrase as cellular automata, built two concrete local-update programs around the exact cyclic and modular anchors above, and benchmarked them against matched same-representation non-CA controls. Only after those branches failed did the repository formalize a literal reserve reading: a pushdown-style prefix filter over autocorrelation debt. That reserve branch was not executed and therefore cannot carry the manuscript’s main claim.

The central difficulty in this setting is methodological rather than rhetorical. Cellular automata, combinatorial-design generation, exact Hadamard constructions, and heuristic Hadamard search

all already have nontrivial literatures [15, 2, 7, 4, 22, 18, 20, 21, 19]. A broad statement such as “cellular automata for Hadamard search is untried” is therefore indefensible. The only fair question is whether a precisely specified local rule on a precisely specified exact object produces a measurable advantage over a matched non-CA comparator. This paper answers that question for the two implemented branches and for no broader family.

Our contributions are:

1. We reconstruct the exact artifacts that define the executed search problems: the 4×79 same-template control, the unresolved length-167, weight-80 cyclic obstruction, the published mod-64 order-668 seed, and the exhaustive structured $n = 9$ control.
2. We formalize two cellular-automata branches and their matched non-CA controls on identical representations, neighborhoods, move caps, budgets, and verifiers.
3. We prove the invariants, exact local-delta formulas, search-space connectivity statements, and fairness properties needed to interpret the comparisons rigorously.
4. We report a reproducible negative result: the implemented H1 branch loses to `direct_greedy` on both its control and target, and the implemented H2 branch fails to beat `direct_greedy` on the only real degraded order-668 start.
5. We separate the validated contribution from the speculative one: the literal “cellar” interpretation survives only as an unexecuted reserve pushdown-style automaton, not as part of the verified claim.

The paper is organized as follows. Section 2 situates the work against exact Hadamard search, CA-based design generation, and automata-side query noise. Section 3 defines the exact objects, metrics, and state spaces. Section 4 gives the algorithms, fairness locks, and the reserve literal-cellar formalization. Section 5 proves the non-obvious mathematical properties of the executed search rules. Sections 6 and 7 describe the experiments and their outcomes. Section 8 states the benchmark limits and the implications for future work.

2 Related Work

2.1 Exact Hadamard anchors and structured search

The executed repository is built around two exact anchors rather than around a free-form search space. The first is the cyclic obstruction isolated by Constantine and Constantine [3]: three fixed binary vectors on $\mathbb{Z}/167\mathbb{Z}$ leave a fourth support of weight 80 whose periodic autocorrelation half-vector must match a specific target. The second is the mod-64 order-668 seed of Eliahou [5], which yields a concrete structured (q, s) representation with a known two-adic quality certificate. We use neither source to claim a new reduction or a new construction; they only define the objects on which the implemented heuristics are evaluated.

At the family and exact-search level, the closest technical comparators are structured Hadamard construction and enumeration methods such as SAT+CAS enumeration for Williamson matrices [2], Williamson-type circulant constructions [7], and Goethals–Seidel difference-family search [4]. These works are important here because they define the novelty boundary. Any alleged cellular-automata

contribution that secretly reduces to family-parameter search, exact pruning, or symmetry-aware exact enumeration is not a new CA method; it is a relabeling of known exact machinery.

Broader modular-Hadamard background is supplied by the survey of Eliahou and Kervaire [6] and the monograph of Horadam [13]. We cite these only for structural context. The present paper does not extend the theory of modular Hadamard matrices; it benchmarks concrete local rules against matched controls on one exact modular seed and one exact cyclic obstruction.

2.2 CA-side prior art and direct heuristic competitors

Cellular automata already appear in combinatorial design and adjacent areas. The survey of Manzoni et al. [15] shows that CA methods are not alien to design generation. Linear cellular automata have been used to construct orthogonal Latin squares [16]; CA heuristics have been applied to bent and semi-bent function search [8, 17]; and controllability or decoder viewpoints provide additional local-dynamics precedents [1, 12]. These works do not solve the present problem, but they are enough to rule out any claim that “CA meets design search” is itself novel.

Direct heuristic search for Hadamard objects is also already on the board. Simulated annealing, simulated quantum annealing, quantum annealing, and QAOA-style formulations have all been proposed for Hadamard search [22, 18, 20, 21, 19]. This makes matched same-representation baselines indispensable: if an executed CA rule does not improve reachability or best-state quality relative to an ordinary local search on the same neighborhood, then the CA label contributes little more than a scheduling choice.

2.3 Lexical watchlist noise and why it matters

Because the initial project prompt was semantically noisy, automated literature search also surfaced papers that are not genuine mathematical comparators, including a shortest-path CA application [23], learning-automata elevator scheduling papers [11, 10], a biomedical CA simulation [9], and an unrelated agents-world conference volume [14]. We retain these only as lexical-noise sentinels: they explain how the seed query drifted, but they do not constrain the technical novelty story. The validated comparison burden lies with exact Hadamard search, structured design construction, CA design generation, and direct Hadamard heuristics, not with papers that share only the word “automata”.

2.4 How this work differs

The executed repository differs from the exact and heuristic literature in one narrow, testable sense only. It fixes the exact cyclic obstruction and the exact modular seed, implements cellular-automata-style local update rules on those objects, and compares them to a matched non-CA local search that uses the same state space and the same verifier. That design is sufficient to support a branch-specific negative result. It is not sufficient to claim a new Hadamard construction, a new cyclic reduction, a new exact solver, a new general CA paradigm for Hadamard search, or a validated literal “cellar” automaton.

3 Background and Preliminaries

3.1 Hadamard and modular-Hadamard context

Definition 3.1 (Hadamard matrix). A Hadamard matrix of order N is a matrix $H \in \{-1, +1\}^{N \times N}$ such that $HH^\top = NI_N$.

Definition 3.2 (Modular Hadamard surrogate). For a positive integer m , an m -modular Hadamard matrix of order N is a sign matrix whose Gram matrix is congruent to NI_N modulo m . The repository's modular branch uses the published 64-modular order-668 seed of Eliahou [5] as a starting point, not as a final Hadamard certificate.

The repository works at the natural block length $n = 167$, since $668 = 4n$. The cyclic branch searches directly over binary supports on $\mathbb{Z}/n\mathbb{Z}$. The modular branch searches over structured sign sequences $(q, s) \in \{-1, +1\}^n \times \{-1, +1\}^n$ from which four length- n sign sequences are derived.

3.2 The exact cyclic obstruction for H1

Let $n = 167$ and let $x \in \{0, 1\}^n$. For $1 \leq k \leq \lfloor n/2 \rfloor$, define the periodic binary autocorrelation half-vector

$$c_x(k) = \sum_{i=0}^{n-1} x_i x_{i+k \bmod n}. \quad (3.1)$$

The executed H1 branch fixes three binary vectors from Constantine and Constantine [3] and searches for a fourth support x of Hamming weight 80 such that

$$c_x(k) = \tau_k \quad \text{for all } 1 \leq k \leq 83, \quad (3.2)$$

where $\tau = (\tau_1, \dots, \tau_{83})$ is the exact target stored in the checked-in H1 target artifact. Figure 2a plots the target half-vector.

Definition 3.3 (H1 objective). For a candidate support x , define the debt vector

$$d_x(k) = c_x(k) - \tau_k, \quad (3.3)$$

the primary objective

$$D_1(x) = \sum_{k=1}^{83} |d_x(k)|, \quad (3.4)$$

and the secondary objective

$$D_\infty(x) = \max_{1 \leq k \leq 83} |d_x(k)|. \quad (3.5)$$

The executed code treats exact hit first, then minimizes (D_1, D_∞) lexicographically.

3.3 The structured modular state for H2

Let $n = 167$ and let $(q, s) \in \{-1, +1\}^n \times \{-1, +1\}^n$. Define the involution-zero transform

$$s_0(i) = \begin{cases} s_i, & 0 \leq i < \lfloor n/2 \rfloor + 1, \\ -s_i, & \lfloor n/2 \rfloor + 1 \leq i < n, \end{cases} \quad (3.6)$$

and let $(sq)_i = s_i q_i$. The repository derives

$$A = s, \quad B = s_0, \quad C = sq, \quad D = (sq)_0. \quad (3.7)$$

For a sign sequence $a \in \{-1, +1\}^n$, define the aperiodic autocorrelation

$$r_a(k) = \sum_{i=0}^{n-1-k} a_i a_{i+k}, \quad 1 \leq k \leq n-1. \quad (3.8)$$

The combined defect vector optimized by the executed H2 branch is

$$g_{q,s}(k) = r_A(k) + r_B(k) + r_C(k) + r_D(k), \quad 1 \leq k \leq 166. \quad (3.9)$$

Definition 3.4 (Two-adic quality metrics). For a nonzero integer z , let $\nu_2(z)$ denote the exponent of the largest power of 2 dividing z . For a defect vector g , define the two-adic modulus

$$M(g) = \min_{k:g(k) \neq 0} 2^{\nu_2(g(k))}, \quad (3.10)$$

with the convention that an exact certificate has no nonzero defects. The repository then ranks states by:

1. exact certificate first;
2. otherwise larger $M(g)$;
3. then smaller ℓ_1 defect $\|g\|_1$;
4. then smaller defect count and smaller maximum defect magnitude.

The published order-668 seed of Eliahou [5] has $M(g) = 64$ and 13 nonzero defect shifts; Figure 2b plots those signed defect values. The real H2 test in this paper begins from a deterministic degradation obtained by flipping the single coordinate s_{41} , which reduces the best available modulus to 16 before search.

3.4 Canonical orbits and fairness accounting

Both branches record orbit statistics under fixed symmetry reductions. The cyclic branch collapses supports under the dihedral action on binary words; the structured branch collapses (g, s) states under signed reversal. These orbit counts do not alter the search rules, but they quantify how much of the state space each method actually visits. This is important because a local method can fail either by converging to poor states or by reaching too few distinct states.

4 Method

4.1 Artifactization and the fairness lock

Table 1 summarizes the four exact artifact packets used by the executed code. The design principle is simple: the comparison must be same-space, same-budget, and same-verifier. For each branch, the serious CA method `parallel_gain_ca` and the serious non-CA comparator `direct_greedy` share the following properties:

Table 1: Exact instances and registered success criteria. The manuscript’s claim is restricted to these four packets.

Packet	Exact object	Representation	Seeds / starts	Success criterion
H1 control	Solved 4×79 control [3]	Weight-43 support on $\mathbb{Z}/79\mathbb{Z}$	8 perturbed + 8 random	Recover the known exact witness
H1 target	Exact 167/80 obstruction [3]	Weight-80 support on $\mathbb{Z}/167\mathbb{Z}$	16 random + 8 projections	Satisfy Eq. (3.2) exactly
H2 ladder	Exhaustive $n = 9$ exact (q, s) control	Structured (q, s) state	3 deterministic one-flip starts	Return to an exact certificate
H2 real seed	Degraded published mod-64 seed [5]	Structured (q, s) with fixed q	1 deterministic degraded start	Improve the best modulus toward 64

1. the same state representation;
2. the same local neighborhood exposed at each step;
3. the same step budget;
4. the same per-phase accepted-move cap;
5. the same exact verifier and ranking objective;
6. the same orbit canonicalization for accounting.

Negative controls `random_rule_ca` and `random_walk` are included to separate genuine objective improvement from mere state-space diffusion.

4.2 H1: support-space cellular automaton on the 167-cycle

Let

$$\mathcal{S}_{n,w} = \left\{ x \in \{0,1\}^n : \sum_{i=0}^{n-1} x_i = w \right\}. \quad (4.1)$$

In H1, $n = 167$ and $w = 80$ on the target, while the control uses $n = 79$ and $w = 43$. The neighborhood is defined by adjacent transpositions along a fixed odd-cycle matching schedule:

$$E^{(1)} = \{(0, 1), (2, 3), \dots\}, \quad (4.2)$$

$$E^{(2)} = \{(1, 2), (3, 4), \dots\}, \quad (4.3)$$

$$E^{(3)} = \{(n-1, 0)\} \quad (4.4)$$

for odd n . At each step only one phase is active.

An edge (a, b) is admissible only when $(x_a, x_b) \in \{(0, 1), (1, 0)\}$, which conserves Hamming weight. For each admissible edge the code computes the exact change in every autocorrelation component and converts it into an exact gain on D_1 . The CA rule then selects locally dominant improving swaps inside a window of radius 2, retaining only moves with gain at least 6, and fires up to four non-conflicting swaps in parallel. The matched baseline `direct_greedy` sees the same candidate set and simply takes the top improving swaps under the same cap.

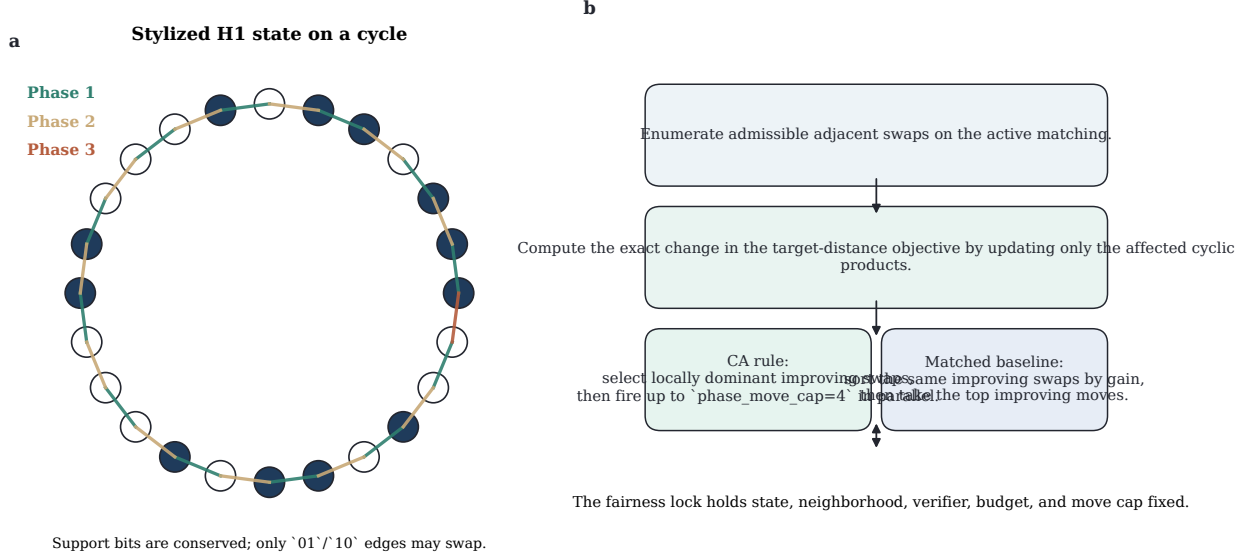


Figure 1: H1 update mechanics. (a) A stylized cycle view of the support state and the three-phase adjacent-swap schedule. Filled nodes indicate support bits equal to one; only `01/10` edges may swap, so weight is conserved exactly. (b) The fairness lock for the serious comparison: both `parallel_gain_ca` and `direct_greedy` enumerate the same admissible swaps and score them with the same exact delta calculation. The only difference is the selection functional, which makes the later empirical comparison interpretable as a same-space test rather than a moving-target benchmark.

4.3 H2: defect-transport CA on the structured (q,s) state

The H2 branch uses even and odd index phases on the line $\{0, \dots, n-1\}$. A single candidate operation flips one coordinate either in s or in q , depending on the declared variable family. On the real order-668 run, q is held fixed and only s flips are allowed. As in H1, both serious methods see the same phase schedule, the same candidate family, and the same cap of four accepted moves per phase.

For each candidate flip, the code computes the exact change in the combined defect vector $g_{q,s}$ from Eq. (3.9). The CA rule keeps locally dominant improving flips inside a radius-1 window and requires gain at least 100 under the scalarized modular objective. The matched baseline `direct_greedy` sorts the same improving flips by gain and applies the best ones under the same cap. Figure 6a summarizes the corresponding structured state.

4.4 Literal “cellar automata” as a reserve, not a validated method

The executed repository eventually formalized a literal reading of “cellar automata” that is materially different from the two cellular-automata branches above. In that reserve concept, a candidate support is parsed as a token word, and a pushdown-style automaton tracks prefix weight, local debt, and a stack of deferred multiscale deficits. A prefix is accepted only if the remaining suffix can still satisfy the exact cyclic target. This reserve branch is mathematically interesting because it changes the state object from a local CA configuration to a constrained prefix language over the exact $167/80$ obstruction. It is not part of the validated contribution, because no transition system,

matched non-CA comparator, or experiment packet exists for it. We therefore discuss it only as future work in Section 8.

4.5 Complexity

Proposition 4.1 (Per-step complexity of H1). *Let n be the support length. One step of either serious H1 method runs in $O(n^2)$ time and $O(n)$ additional memory.*

Proof. The active matching contains $O(n)$ edges. For each admissible edge (a, b) , the exact delta computation loops over $k = 1, \dots, \lfloor n/2 \rfloor$, and for each k it evaluates only a constant-size set of start positions. Thus each candidate edge is processed in $O(n)$ time and $O(n)$ temporary space for the stored half-vector delta. Summed over $O(n)$ edges, this gives $O(n^2)$ total time. The remaining operations in a step — sorting or filtering candidates, applying at most four swaps, updating the state, and recording snapshots — are lower-order compared with the candidate-generation pass. Memory beyond the stored state consists of the candidate list and temporary deltas, both bounded by $O(n)$. $\square$

Proposition 4.2 (Per-step complexity of H2). *Let n be the structured state length. One step of either serious H2 method runs in $O(n^2)$ time and $O(n)$ additional memory.*

Proof. Each parity phase exposes $O(n)$ indices. For one candidate flip at index j , the exact delta routine loops over shifts $k = 1, \dots, n - 1$ and, for each shift, updates only a constant number of terms and a constant number of affected sequences. Hence one candidate costs $O(n)$ time and $O(n)$ temporary space. Over $O(n)$ candidate indices, the phase costs $O(n^2)$ time. As in H1, later selection and state-update operations are lower-order. $\square$

5 Theoretical Results and Proofs

5.1 The H1 state space

Proposition 5.1 (Weight preservation and connectivity in H1). *Fix integers $n \geq 2$ and $0 \leq w \leq n$. Every executed H1 move preserves the state space $\mathcal{S}_{n,w}$. Moreover, the graph on $\mathcal{S}_{n,w}$ whose edges are adjacent transpositions of coordinates $(i, i + 1)$ for $0 \leq i \leq n - 2$ is connected, and therefore the larger cycle-neighborhood graph used by H1 is also connected.*

Proof. For preservation, consider one executed swap on adjacent coordinates (a, b) . By construction, the move is executed only when $(x_a, x_b) \in \{(0, 1), (1, 0)\}$. After the swap, the ordered pair becomes $(1, 0)$ or $(0, 1)$, so the sum of the two entries remains 1. All other coordinates are unchanged. Hence $\sum_i x_i$ is preserved, which proves that x remains in $\mathcal{S}_{n,w}$.

For connectivity, it suffices to work with the path-adjacent transpositions $(0, 1), (1, 2), \dots, (n-2, n-1)$ because these moves are available inside the larger cycle graph. Those adjacent transpositions generate the full symmetric group S_n . Let $x, y \in \mathcal{S}_{n,w}$. Because x and y have the same Hamming weight, there exists a permutation $\pi \in S_n$ that sends the support of x to the support of y . Since π can be written as a product of path-adjacent transpositions, applying that product to the coordinates of x transforms x into y . Some transpositions may swap equal bits and therefore leave the current state unchanged, but this does not invalidate the path: each adjacent transposition in the factorization is

an allowed edge in the state graph, and the resulting sequence ends at y . Therefore the path-adjacent transposition graph is connected, and the cycle-neighborhood graph used by H1 is connected as well. $\square$

Lemma 5.2 (Exact local delta for a single H1 swap). *Let $x \in \{0, 1\}^n$, let (a, b) be an adjacent edge with $a \neq b$, and let x' be the vector obtained by swapping x_a and x_b . For a fixed shift k , define*

$$S_{a,b}(k) = \{a, b, a - k, b - k\} \subset \mathbb{Z}/n\mathbb{Z}. \quad (5.1)$$

Then

$$c_{x'}(k) - c_x(k) = \sum_{i \in S_{a,b}(k)} (x'_i x'_{i+k} - x_i x_{i+k}). \quad (5.2)$$

Proof. By definition,

$$c_x(k) = \sum_{i \in \mathbb{Z}/n\mathbb{Z}} x_i x_{i+k}. \quad (5.3)$$

The two vectors x and x' differ only at coordinates a and b . Therefore the summand indexed by i can change only if at least one of the two factors uses coordinate a or coordinate b . The first factor uses a or b exactly when $i \in \{a, b\}$. The second factor uses a or b exactly when $i + k \in \{a, b\}$, or equivalently $i \in \{a - k, b - k\}$ in $\mathbb{Z}/n\mathbb{Z}$. Thus the only indices whose summands can change are those in $S_{a,b}(k)$.

If $i \notin S_{a,b}(k)$, then neither i nor $i + k$ equals a or b , so both factors are unchanged and

$$x'_i x'_{i+k} = x_i x_{i+k}. \quad (5.4)$$

Subtracting the two full sums, every unchanged summand cancels. The remaining summands are exactly those indexed by $S_{a,b}(k)$, which proves Eq. (5.2). If the set definition introduces duplicates modulo n , the duplicate indices still contribute only once because the original full sum also contains each residue class only once. $\square$

Proposition 5.3 (Candidate-set identity in H1). *At every step of the executed H1 comparison, `parallel_gain_ca` and `direct_greedy` evaluate the same candidate edge set, use the same exact gain values, and are constrained by the same accepted-move cap. Their only difference is the selection functional applied to the shared candidate list.*

Proof. The active phase determines a fixed matching E_t . Both methods inspect exactly the admissible edges in E_t , namely those edges whose endpoint bits differ. By Lemma 5.2, each candidate edge has a well-defined exact delta in the autocorrelation half-vector and therefore a well-defined exact gain on the primary objective D_1 . The repository code uses the same delta routine and the same objective function for both methods. The parameter `phase_move_cap=4` is also shared.

What differs is the final selection step. `parallel_gain_ca` keeps locally dominant improving candidates inside a fixed radius-2 window, while `direct_greedy` sorts all improving candidates by gain and keeps the best ones. No other part of the comparison changes. Hence the two methods are same-space, same-budget, same-verifier comparators whose only algorithmic difference is move selection. $\square$

5.2 The H2 state space

Proposition 5.4 (Connectivity of fixed-family H2 search). *Fix $n \geq 1$ and either a fixed $q \in \{-1, +1\}^n$ with mutable $s \in \{-1, +1\}^n$, or a fixed $s \in \{-1, +1\}^n$ with mutable $q \in \{-1, +1\}^n$. Under single-coordinate flips of the mutable family, the corresponding state graph is connected.*

Proof. Consider the case of fixed q and mutable s ; the fixed- s case is identical. The state space is exactly $\{-1, +1\}^n$, indexed by the mutable vector s . Two states are adjacent when they differ in exactly one coordinate. That graph is the n -dimensional hypercube, which is connected. Explicitly, for any two states s and s' , flip every coordinate i for which $s_i \neq s'_i$. After finitely many such single-coordinate flips the mutable vector equals s' , so the graph is connected. $\square$

Lemma 5.5 (Exact local delta for a single H2 flip). *Fix a variable family s or q , fix an index j , and let (q', s') be obtained from (q, s) by flipping the chosen coordinate in the chosen family. For a fixed shift k , the change in $g_{q,s}(k)$ is the sum of the changed correlation terms with one endpoint at j over the affected derived sequences. More precisely, if $\mathcal{F}$ is the affected label set $(\{A, B, C, D\})$ for an s -flip and $\{C, D\}$ for a q -flip, then*

$$g_{q',s'}(k) - g_{q,s}(k) = \sum_{L \in \mathcal{F}} \left(\mathbf{1}_{j+k < n} (a_j^{(L)'} a_{j+k}^{(L)'} - a_j^{(L)} a_{j+k}^{(L)}) + \mathbf{1}_{j-k \geq 0} (a_{j-k}^{(L)'} a_j^{(L)'} - a_{j-k}^{(L)} a_j^{(L)}) \right), \quad (5.5)$$

where $a^{(L)}$ denotes the sequence attached to label L and the indicator functions suppress nonexistent aperiodic terms.

Proof. For one label L , the shift- k contribution to $g_{q,s}(k)$ is

$$r_{a^{(L)}}(k) = \sum_{i=0}^{n-1-k} a_i^{(L)} a_{i+k}^{(L)}. \quad (5.6)$$

After flipping coordinate j in the chosen variable family, every derived sequence outside the affected label set $\mathcal{F}$ is unchanged, so only $L \in \mathcal{F}$ matter.

Fix such a label L . The summand indexed by i changes only if one of the two factors uses coordinate j . In the forward case this means $i = j$, which is present only when $j + k < n$. In the backward case this means $i + k = j$, equivalently $i = j - k$, which is present only when $j - k \geq 0$. For any other index i , neither factor uses the flipped entry and the summand is unchanged. Therefore the total change in $r_{a^{(L)}}(k)$ is exactly the sum of the two displayed terms when they exist. Summing over all affected labels $L \in \mathcal{F}$ proves Eq. (5.5). $\square$

Proposition 5.6 (Candidate-set identity in H2). *At every step of the executed H2 comparison, `parallel_gain_ca` and `direct_greedy` evaluate the same candidate flip set, use the same exact modular objective, and are subject to the same accepted-move cap. Their only difference is the selection functional.*

Proof. The active parity phase determines the same mutable index set for both methods. The declared variable family is fixed for the run, so the candidate operations are identical. By Lemma 5.5, each candidate operation has a well-defined exact change in the combined defect vector and hence a

well-defined exact change in the modular objective used by both methods. The per-phase cap is shared: both serious methods apply at most four candidate flips in one phase.

The only difference appears after candidate evaluation. `parallel_gain_ca` retains locally dominant improving flips inside a radius-1 window and requires gain at least 100, while `direct_greedy` sorts all improving flips and keeps the best ones. No representation, neighborhood, objective, or budget advantage is granted to either method. $\square$

Remark 5.7. Propositions 5.3 and 5.6 are the theoretical basis for the empirical interpretation in Sections 7. They show that the decisive comparisons are not confounded by hidden differences in search space, verifier strength, or budget accounting. When Figure 4 shows a loss and Figure 6 shows a tie, the result is attributable to the move-selection rule rather than to a benchmark mismatch.

6 Experimental Setup

6.1 Software, hardware, and reproduction commands

The checked-in code is organized as follows: artifact construction in `hadamard668/artifacts.py`, cyclic search in `hadamard668/h1.py`, structured modular search in `hadamard668/h2.py`, verifier logic in `hadamard668/verifiers.py`, and the experiment runner in `hadamard668/experiments.py`. The canonical reproduction commands are:

```
python3 -m hadamard668.artifacts export
python3 -m hadamard668.experiments run-h1
python3 -m hadamard668.experiments run-h2
python3 -m hadamard668.experiments run-all
```

For this writer pass we additionally reran

```
python3 -m hadamard668.artifacts export --output-dir /tmp/h668_artifacts_check
python3 -m hadamard668.experiments run-all --output-dir /tmp/h668_repro_check
```

and verified that the four artifact JSON files reproduced byte-for-byte, while the experiment summaries matched exactly after excluding serialized wall-clock fields. The current environment reports Python 3.10.17, Linux 4.4.0 on x86_64, 20 virtual CPUs, and 448 GiB of RAM. The original packet did not serialize CPU model information for the earlier runs, so we report the current rerun environment explicitly and treat the missing original hardware metadata as a benchmark limitation rather than silently fabricating it.

6.2 Hyperparameters, budgets, and seeds

The hyperparameters are fully fixed in code. For H1, `parallel_gain_ca` uses window = 2, minimum gain = 6, accepted-move cap = 4, and snapshot interval = 12; the control and target sweeps each use 96 steps. For H2, `parallel_gain_ca` uses window = 1, minimum gain = 100, accepted-move cap = 4, and snapshot interval = 8; the ladder uses 24 steps and the real order-668 run uses 48 steps. The negative controls keep the same move cap and use the same snapshot cadence. Table 2 records the settings compactly.

Table 2: Locked hyperparameters and budgets for the executed packet. No ablation matrix exists beyond these settings.

Branch / method	Window	Min gain	Phase move cap	Steps
H1 parallel_gain_ca	2	6	4	96
H1 direct_greedy	—	improving only	4	96
H1 random_rule_ca	—	stochastic fire prob. 0.2	4	96
H1 random_walk	—	one random admissible move	4	96
H2 parallel_gain_ca	1	100	4	24 / 48
H2 direct_greedy	—	improving only	4	24 / 48
H2 random_rule_ca	—	stochastic fire prob. 0.25	4	24 / 48
H2 random_walk	—	one random admissible move	4	24 / 48

The seed schedule is also deterministic. The H1 control packet uses eight perturbed-solution starts with perturbation counts $2, 4, \dots, 16$ and eight random-weight supports with seeds 20000 through 20007. The H1 target packet uses sixteen random-weight supports with seeds 30000 through 30015 together with eight deterministic modular projections of the published mod-64 seed under shifts 0 through 7. The H2 ladder uses three deterministic one-flip starts obtained from an exhaustive $n = 9$ exact seed. The real H2 packet uses one deterministic s_{41} flip of the published order-668 seed.

6.3 Metrics and statistical treatment

For H1 the decisive metric is the best achieved value of D_1 , with exact-hit rate and D_∞ as supporting metrics. For H2 the decisive metrics are the best achieved modulus $M(g)$, the best ℓ_1 defect, defect count, and maximum defect magnitude, together with exact-hit or goal-hit indicators depending on the control mode. Orbit counts, accepted moves, and best-step timing are recorded for all runs.

Because the runs are deterministic conditional on the seed schedule, the H1 error bars in this paper are uncertainty summaries over the finite seed panel rather than stochastic optimization noise. We use nonparametric bootstrap 95% confidence intervals for H1 mean best distance and for the displayed H1 orbit summaries. For the $n = 9$ H2 ladder we report Wilson 95% intervals on the binomial goal-hit rate. For the single real H2 start, no interval estimate is appropriate; we report the deterministic values directly and state the $N = 1$ limitation explicitly.

7 Results

7.1 Problem objects and overall program flow

Figure 2 displays the exact target objects. The cyclic obstruction is not a vague scoring function; it is a fixed target vector τ of length 83. Likewise, the mod-64 seed is not a generic near-solution; it has a concrete signed defect spectrum and a known best modulus. Figure 3 then shows the executed research flow. The important point is negative: the manuscript’s validated claim comes entirely from the executed H1 and H2 branches. The reserve literal-cellar branch is visibly outside the empirical packet.

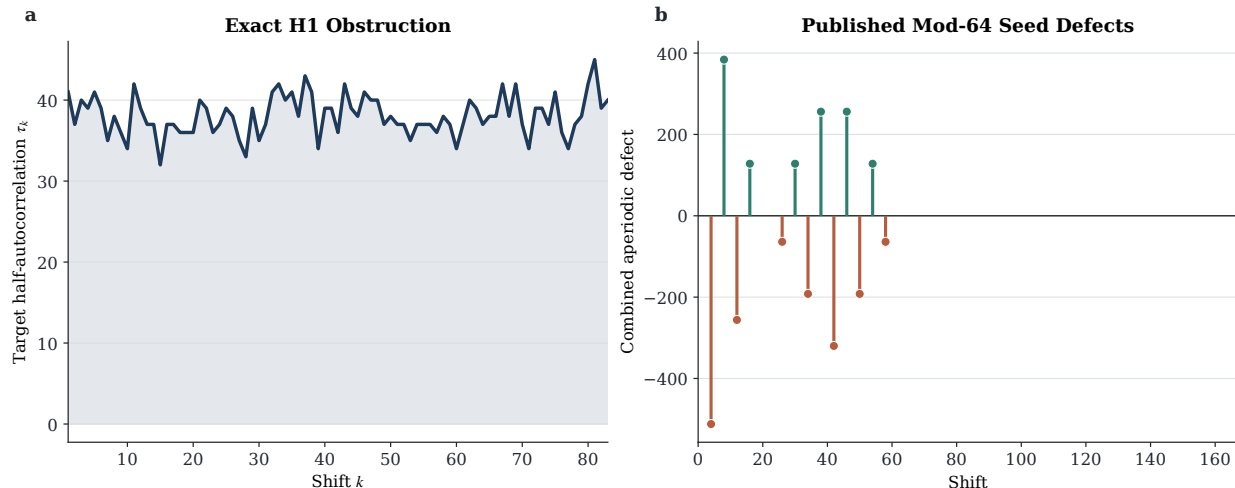


Figure 2: Exact objects that define the search problem. (a) The H1 target half-autocorrelation vector τ from the unresolved length-167, weight-80 cyclic obstruction of Constantine and Constantine [3]. The branch objective is the exact ℓ_1 distance to this vector, not an approximate proxy. (b) The signed nonzero combined aperiodic defects of the published mod-64 order-668 seed from Eliahou [5]. The best available modulus is 64 before degradation; the real H2 test begins only after a deterministic s_{41} flip reduces that modulus to 16.

7.2 H1 fails on both the control and the target

Table 3 and Figure 4 summarize the decisive H1 packet. On the solved 4×79 control, neither serious method recovers an exact witness under the locked budget. Even so, the matched baseline `direct_greedy` is better on the primary metric: its best-distance mean is 41.5 (bootstrap 95% CI [37.9, 45.0]), versus 48.375 (CI [43.3, 54.5]) for `parallel_gain_ca`. On paired seeds, `direct_greedy` wins 11/16 comparisons on the primary metric. There is one primary tie, and on that tie the secondary metric D_∞ still favors `direct_greedy`, so the full objective tuple effectively favors `direct_greedy` in 12/16 cases. The control therefore ranks the two serious methods but does not validate solver competence, because exact-hit rate remains 0/16 for both.

On the exact 167/80 target, the result is more decisive. Neither method finds a witness in 24 starts, but `direct_greedy` beats `parallel_gain_ca` on all 24/24 matched seeds. The best-distance mean is 147.9 (CI [141.8, 154.3]) for `direct_greedy` and 188.5 (CI [178.2, 199.4]) for `parallel_gain_ca`. Median unique dihedral orbits also favor the baseline: 23.5 versus 7.5. Figure 5 shows that the CA branch typically reaches its best recorded state much earlier than the baseline, which is consistent with premature stagnation rather than with a delayed but stronger search dynamic.

7.3 H2 supplies no advantage on the real order-668 task

The H2 ladder and real-seed results are reported in Table 4 and Figure 6. The $n = 9$ ladder is not a convincing positive control. The CA solves 2/3 starts and the matched baseline solves 1/3, but `random_rule_ca` solves all three starts and `random_walk` solves two. Random controls therefore perform too well for the ladder to discriminate a genuine CA mechanism from a generally easy toy problem.

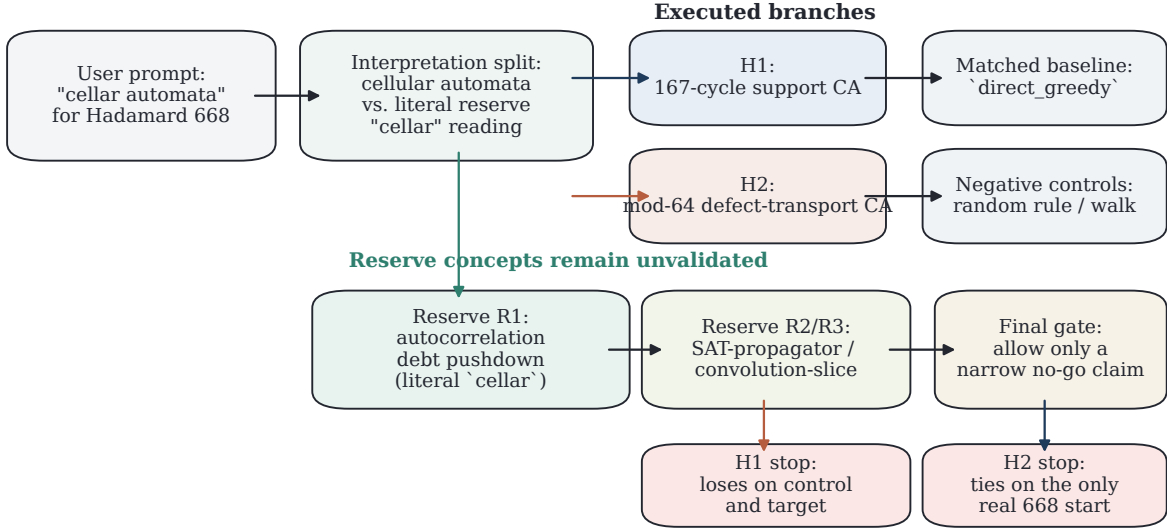


Figure 3: Program flow from the original prompt to the validated claim. The executed packet first interprets “cellar automata” as cellular automata and instantiates two branches: H1 on the exact 167/80 cyclic obstruction and H2 on the published mod-64 seed. Each branch is paired with a matched same-space non-CA baseline and with two negative controls. Both executed branches fail the registered gates, so the final evidence gate allows only a narrow no-go claim. The literal-cellar pushdown concept remains a reserve branch with no experimental credit.

The decisive H2 test is the real degraded order-668 seed. Here the comparison is deterministic and exact: both `parallel_gain_ca` and `direct_greedy` achieve best modulus 16, best ℓ_1 defect 2944, defect count 25, maximum defect magnitude 496, accepted moves 48, best step 2, and only two unique orbits. Both methods finish back at ℓ_1 defect 3200. The random controls eventually attain lower final ℓ_1 defect, but only by dropping the modulus from 16 to 8, which makes them inferior under the registered modular objective. Thus the real task produces no CA advantage at all: the serious methods tie on the only condition that matters for the executed branch.

7.4 Reproducibility and the exact meaning of the no-go claim

The validated claim is narrow and exact:

For the checked-in H1 and H2 rules, on the exact artifacts and budgets tested here, the implemented cellular automata do not outperform their matched non-CA baselines, and they produce no order-668 solution.

The phrase “matched” is load-bearing. Propositions 5.3 and 5.6 show that the decisive comparisons are apples-to-apples in the precise sense of same candidate set, same verifier, and same cap. The manuscript therefore does not need to speculate about whether a hidden budget or representation advantage explains the losses: it does not.

This same narrowness also determines what the paper does *not* prove. It does not show that all possible CA-like search strategies for Hadamard order 668 are doomed. It does not show that a

Table 3: H1 results on the same-template control and the exact 167/80 target. Confidence intervals are bootstrap 95% intervals over the deterministic seed panel.

Packet	Method	Hit rate	Best-dist. mean	95% CI	Median orbits
H1 control	direct_greedy	0/16	41.5	[37.9, 45.0]	13.0
H1 control	parallel_gain_ca	0/16	48.375	[43.3, 54.5]	4.0
H1 control	random_rule_ca	0/16	57.125	—	66.5
H1 control	random_walk	0/16	58.25	—	78.5
H1 target	direct_greedy	0/24	147.9	[141.8, 154.3]	23.5
H1 target	parallel_gain_ca	0/24	188.5	[178.2, 199.4]	7.5
H1 target	random_rule_ca	0/24	225.4	—	67.5
H1 target	random_walk	0/24	253.6	—	78.0

Table 4: H2 results. The ladder goal-hit intervals are Wilson 95% intervals. The real order-668 packet has $N = 1$, so the deterministic values are reported directly.

Packet	Method	Hit rate	Best ℓ_1 defect	Best mod.	Median orbits
H2 ladder	parallel_gain_ca	2/3	2.67 mean	8	3
H2 ladder	direct_greedy	1/3	10.67 mean	8	3
H2 ladder	random_rule_ca	3/3	0 mean	variable	18
H2 ladder	random_walk	2/3	5.33 mean	variable	23
H2 real seed	parallel_gain_ca	0/1	2944 best / 3200 final	16 best	2
H2 real seed	direct_greedy	0/1	2944 best / 3200 final	16 best	2
H2 real seed	random_rule_ca	0/1	3200 best / 2056 final	16 best / 8 final	49
H2 real seed	random_walk	0/1	3200 best / 2256 final	16 best / 8 final	49

literal pushdown-style cellular automaton cannot help. It does not even show that all local-repair views of the mod-64 seed are noncompetitive. It shows only that the specific same-space local-dominance rules implemented here are not competitive against the matched direct greedy baseline.

8 Discussion

8.1 What failed and why that failure is informative

The executed program fails for two different but related reasons. In H1, the CA rule explores fewer unique dihedral orbits and reaches its best recorded state earlier than the matched baseline. The most honest interpretation is that the local-dominance rule is too conservative: parallelism does not buy new reachability, and the gain threshold accelerates stagnation. In H2, the local-dominance rule is not obviously worse than the matched baseline on the toy ladder, but the only real order-668 start collapses the distinction completely. The resulting tie shows that, once the representation, neighborhood, and objective are fixed, the CA label does not by itself create a repair mechanism.

This is why the negative result is scientifically useful. The repository does not merely fail to solve order 668; it identifies a concrete reason to stop spending more compute on the current rule family. A larger budget on the same H1 adjacent-swap rule or the same H2 s -flip rule would be hard to justify when the current packet already shows a control loss, a target loss, and a real-task tie under

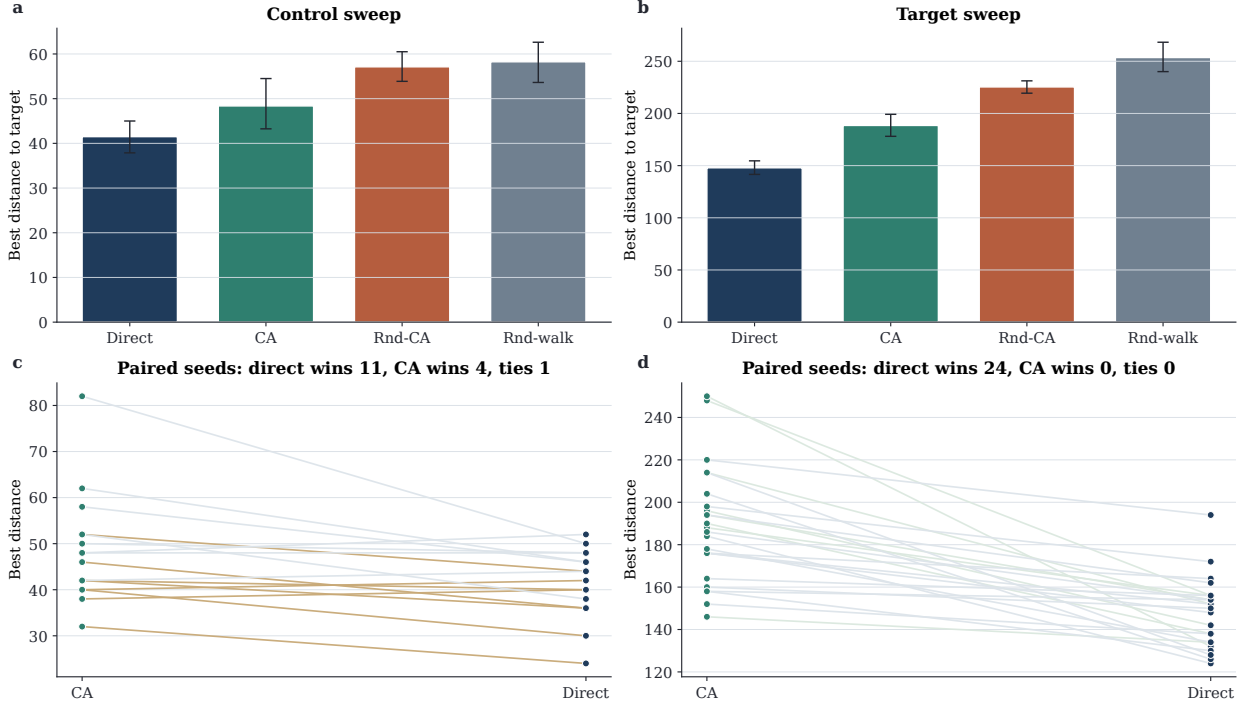


Figure 4: Main H1 quantitative comparisons. Top row: mean best distance to the exact target with bootstrap 95% confidence intervals for the control and the target sweeps. Bottom row: paired seed-level comparisons between `parallel_gain_ca` and `direct_greedy`. On the control, the baseline wins a clear majority of seeds even though neither method solves exactly. On the target, every paired comparison favors `direct_greedy` on the primary metric. Because Propositions 5.1–5.3 show that the state space, neighborhood, objective, and move cap are shared, these losses cannot be explained away as benchmark mismatch.

matched conditions.

8.2 Benchmark limits

The verification gate imposes several limitations that must be stated explicitly.

1. **The H1 control is solved in principle but not solved by the locked stack.** Both serious methods have exact-hit rate $0/16$ on the same-template 4×79 control. The control ranks heuristics, but it does not certify that the evaluation stack is itself strong enough to recover a known witness under the published budget.
2. **The H2 real task is underpowered.** The validated packet contains only one degraded order-668 start, with q fixed and only s mutable. That is enough to reject the executed branch, not enough to generalize about all structured repair variants.
3. **No CA ablation matrix exists.** Each branch is evaluated at one CA setting only. The paper therefore rejects an implemented rule family, not an entire hyperparameter envelope.
4. **Only one serious non-CA baseline is present.** The comparison is fair but thin. A

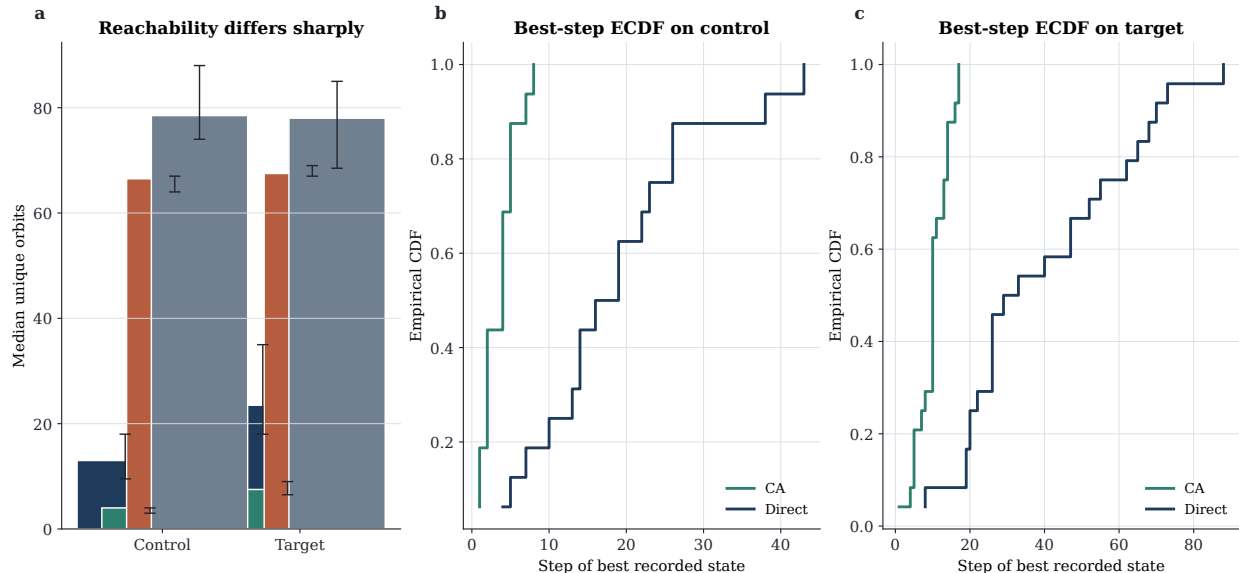


Figure 5: H1 reachability and timing diagnostics. Panel (a) reports orbit summaries for both packets: the serious CA rule visits far fewer canonical orbits than the matched baseline, while the negative controls diffuse broadly but do not achieve good objective values. Panels (b)–(c) show empirical CDFs of the step at which the best recorded state is first achieved. The CA branch peaks much earlier than the baseline on both packets, especially on the exact target, which is consistent with the repository’s interpretation of a reachability collapse rather than with a hidden long-horizon advantage.

stronger publication-grade benchmark would add at least one more same-representation heuristic baseline.

5. **The original run packet under-serializes some fairness details.** The matching code and benchmark sheet carry part of the audit burden because the baseline JSON records fewer method parameters than the CA JSON. We preserved the narrow claim rather than overstating the auditability.

8.3 The status of the literal “cellar” branch

The literal reserve branch is summarized visually in Figure 7. It is not another one-dimensional CA with a renamed neighborhood. Its intended state is a prefix language with deferred autocorrelation debt. That idea survives conceptually because it changes the search object from local support bits to extendable prefix states. Yet it remains scientifically unvalidated. No transition system has been implemented, no matched non-automaton comparator exists, and no exact positive control has been preserved in that representation. Any future paper about literal cellar automata would therefore need a new design brief, a new matched baseline, and a separate benchmark packet.

8.4 Societal and scientific implications

The immediate societal stakes are low: this paper is about negative evidence in pure-math search. The scientific-process stakes are less trivial. AI-assisted search can make a weak program look

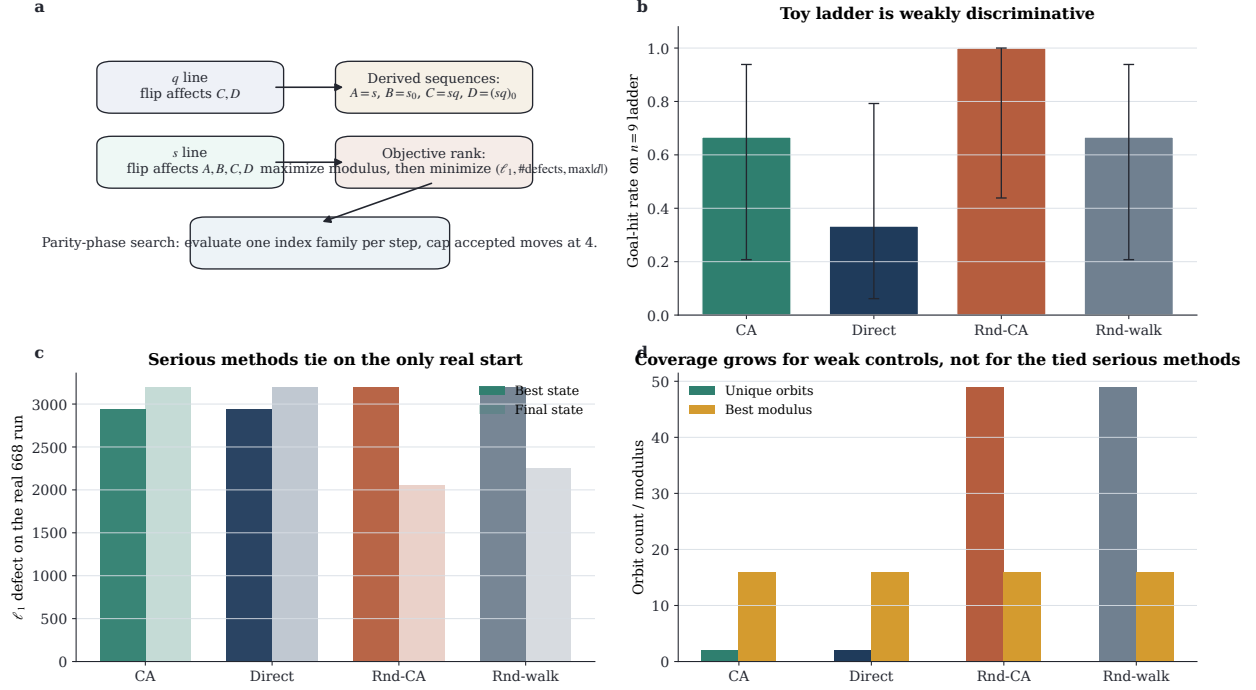


Figure 6: H2 mechanism and results. (a) The structured (q, s) state induces four derived sequences whose combined aperiodic defect vector defines the modular objective. (b) The toy $n = 9$ ladder is a weak discriminator because random controls solve most starts; the wide Wilson intervals reflect the tiny panel size. (c) On the only real degraded order-668 start, `parallel_gain_ca` and `direct_greedy` tie exactly on the best attained ℓ_1 defect and both regress to the same final state. (d) Weak controls visit many more orbits, but that coverage is not useful because it is achieved by sacrificing the registered modular objective.

stronger than it is by changing terminology faster than it changes mathematics. In that sense, the most useful outcome here is methodological restraint. By forcing the comparison to remain same-space and same-budget, the repository avoids a common failure mode in automated discovery systems: mistaking a repackaged local search for a new algorithmic principle. Negative results that are cleanly benchmarked can save substantial downstream compute and reduce the chance that speculative labels such as “cellar automata” acquire undeserved evidentiary force.

9 Conclusion

The executed repository does not solve the Hadamard order-668 problem, and it does not validate a cellular-automata route to that solution. What it does provide is a reproducible, branch-specific no-go result. The H1 support-space CA loses to its matched same-representation baseline on both a solved control and the exact 167/80 target. The H2 defect-transport CA fails to beat the same matched baseline on the only real degraded order-668 seed that was tested. Because the benchmark fairness properties are explicit and because the exact target objects are fixed by primary sources, this negative result is stronger than a generic “we did not find a solution” report.

The literal “cellar automata” interpretation survives only as a reserve pushdown-style idea, not as

Executed and reserve branches separate cleanly

H1 executed	yes	yes	yes	no	no	no
H2 executed	yes	yes	yes	no	no	no
R1 cellar / pushdown	yes	no	no	no	no	yes
R2 SAT-propagator	no	no	no	no	no	yes
R3 convolution-slice	yes	no	no	no	no	yes
	Exact anchor	Executed	Matched baseline	Positive result	Validated contribution	Reserve only

Figure 7: Executed branches versus reserve branches. Both executed branches have exact anchors, were run under matched baselines, and produced no positive result. The literal cellar / pushdown reserve and the convolution-slice reserve remain interesting only because they change the state representation materially. They do not inherit any empirical credit from the failed H1/H2 packet.

an executed contribution. Any future attempt to reopen the project should therefore start from a materially different state representation, preserve same-space positive controls, and benchmark against a matched non-automaton comparator from the beginning. Under the current evidence gate, more budget on the present CA rules would be a mistake.

A Extended Proofs and Formal Clarifications

A.1 Why the H1 primary tie still favors the baseline

On the H1 control packet, one paired seed produces a tie on the primary metric D_1 : the random-weight seed with deterministic seed 20002 yields best $D_1 = 48$ for both `parallel_gain_ca` and `direct_greedy`. The secondary metric D_∞ then breaks the tie in favor of `direct_greedy`, which attains $D_\infty = 4$ versus $D_\infty = 5$ for `parallel_gain_ca`. This is why the control can be summarized as 11/16 primary wins for the baseline but effectively 12/16 wins on the full objective tuple.

A.2 Why the H2 random controls are not counterexamples

On the real H2 packet, the negative controls eventually end with smaller final ℓ_1 defect than the serious methods. This is not a contradiction of the registered conclusion because the objective is not “minimize final ℓ_1 defect at all costs.” The serious methods preserve best modulus 16 and improve the best state only transiently; the random controls finish with modulus 8, which is strictly worse under the lexicographic modular objective. The repository therefore treats those trajectories as evidence of metric leakage risk, not as serious wins.

B Additional Experimental Detail

B.1 Exact packet sizes and positive-control facts

The control and target artifacts have the following exact combinatorial signatures:

- The 4×79 control uses three fixed vectors of weights $(34, 34, 42)$ and one known solution vector of weight 43.
- The exact H1 target uses three fixed vectors of weights $(76, 76, 77)$ and an unresolved fourth support of required weight 80.
- The published H2 seed has 13 nonzero combined defect shifts and best modulus 64 before degradation.
- The structured $n = 9$ H2 control was found by exhaustive search over all $512 \times 512 = 262144$ sign pairs.

B.2 Real H2 trade-off diagnostics

The single real order-668 start records the following best-versus-final states:

Method	Best modulus	Best ℓ_1 defect	Final modulus	Final ℓ_1 defect
parallel_gain_ca	16	2944	16	3200
direct_greedy	16	2944	16	3200
random_rule_ca	16	3200	8	2056
random_walk	16	3200	8	2256

These values explain the interpretation used in the main text: the serious methods tie on the best state, while the random controls only improve one metric by losing the more important modulus objective.

C Reproducibility Notes

All manuscript figures were generated programmatically by

```
python3 figures/generate_figures.py
```

which writes both PDF and PNG files to `figures/` and writes the derived numerical summaries to `results/analysis/paper_metrics.json`. During manuscript preparation we also reran the artifact export and full experiment stack into temporary directories. The artifact JSON files matched byte-for-byte. The experiment JSON files differed at the raw file level because they include wall-clock fields, but all method summaries matched exactly after removing the elapsed-time keys.

D Hyperparameter Sensitivity Debt

This appendix exists because a real hyperparameter sensitivity analysis was *not* run. The executed packet tests one CA setting per branch only. We therefore separate observed single-setting diagnostics from unmeasured sensitivity:

1. **Observed under the fixed H1 setting.** The CA branch reaches its best recorded state with median step 4 on the control and median step 10 on the target, far earlier than the matched baseline. This is consistent with early plateauing.
2. **Observed under the fixed H2 setting.** On the real order-668 start, both serious methods reach their best state at step 2 and then regress, which suggests that the current acceptance rule does not stabilize gains.
3. **Not measured.** No grid over window size, minimum gain, move cap, or budget was executed. Any broader performance claim would therefore be unjustified.

The correct scientific interpretation is not that all nearby hyperparameters have been falsified, but that the currently implemented rule families have failed strongly enough that a follow-on study should change representation before it changes budget.

References

- [1] Franco Bagnoli, Sana Dridi, and Nazim Fatès. Regional controllability of cellular automata as a sat problem. *Natural Computing*, 2025. doi: 10.1007/s11047-025-10061-6. URL <https://www.semanticscholar.org/paper/d622c453cb5aa83cdcd35eb077178d61c7bfcaf>.
- [2] Curtis Bright, I. Kotsireas, and Vijay Ganesh. A sat+cas method for enumerating williamson matrices of even order. *AAAI Conference on Artificial Intelligence*, 2018. doi: 10.1609/aaai.v32i1.12203. URL <https://www.semanticscholar.org/paper/eec5b2d83d3853c24cf070b4701fc2c86b594222>.
- [3] Gregory M. Constantine and Rodica R. Constantine. Convolution numbers: the cyclic case. *arXiv*, 2025. URL <https://arxiv.org/abs/2501.18066>.
- [4] Dragomir Z. Dokovic and Ilias Kotsireas. Goethals–seidel difference families with symmetric or skew base blocks. *Mathematics and Computer Science*, 2018. URL <https://www.semanticscholar.org/paper/09ffbc2db12a717278cb6ab7782bdc54d975138b>.
- [5] Shalom Eliahou. A 64-modular hadamard matrix of order 668. *Australasian Journal of Combinatorics*, 93(2):422–429, 2025. URL https://ajc.maths.uq.edu.au/pdf/93/ajc_v93_p422.pdf.
- [6] Shalom Eliahou and Michel Kervaire. A survey on modular hadamard matrices. *Discrete Mathematics*, 302(1–3):85–106, 2005. doi: 10.1016/j.disc.2004.08.013. URL <https://www.semanticscholar.org/paper/b0bc1d196cbba14824aebcbb2410226ff2688d3d>.
- [7] Patrick Fitzpatrick and Henry O’Keeffe. Williamson type hadamard matrices with circulant components. *Discrete Mathematics*, 346(12):113615, 2023. doi: 10.1016/j.disc.2023.113615. URL <https://www.sciencedirect.com/science/article/pii/S0012365X23003011>.

- [8] Maxime Gadouleau, Luca Mariot, and Stjepan Picek. Bent functions from cellular automata. *IACR Cryptology ePrint Archive*, 2020. URL <https://eprint.iacr.org/2020/1272>.
- [9] M. Ghaemi, Mina Shojafar, Zahra Zabihipour, and Y. Asgari. On the possibility of oscillating in the ebola virus dynamics and investigating the effect of the lifetime of t lymphocytes. *PLoS ONE*, 2022. doi: 10.1371/journal.pone.0265065. URL <https://www.semanticscholar.org/paper/4c744f3e519204bac75f7a99661bbe3491393fc1>.
- [10] Omar Ghaleb and B. Oommen. Learning automata-based solutions to the multi-elevator problem. *International Conference on Intelligent Computing*, 2019. doi: 10.1007/978-3-030-26766-7_13. URL <https://www.semanticscholar.org/paper/9a8e718e861bc6a07be3b5cee1fa506c7f413b3d>.
- [11] Osamah Ghaleb and B. Oommen. Learning automata-based solutions to the single elevator problem. *Artificial Intelligence Applications and Innovations*, 2019. doi: 10.1007/978-3-030-19823-7_37. URL <https://www.semanticscholar.org/paper/74159b79e948de4dd6fab33ecd01d4a0c96a51c5>.
- [12] Michael Herold, Earl T. Campbell, Jens Eisert, and Michael J. Kastoryano. Cellular-automaton decoders for topological quantum memories. *npj Quantum Information*, 2015. doi: 10.1038/npjqi.2015.10. URL <https://www.nature.com/articles/npjqi201510>.
- [13] Kathy J. Horadam. *Hadamard Matrices and Their Applications*. Princeton University Press, 2007. ISBN 9780691125776. URL https://books.google.com/books/about/Hadamard_Matrices_and_Their_Applications.html?id=4tmaMU4yW6AC.
- [14] J. Leeuwen, Andrea Omicini, R. Tolksdorf, and F. Zambonelli. Engineering societies in the agents world. In *Engineering Societies in the Agents World*, 2000. doi: 10.1007/3-540-44539-0. URL <https://www.semanticscholar.org/paper/094836f18350868a0b8cfe57a127906496120841>.
- [15] Luca Manzoni, Luca Mariot, and Giuliamaria Menara. Combinatorial designs and cellular automata: A survey. *Discrete Applied Mathematics*, 2025. doi: 10.48550/arXiv.2503.10320. URL <https://www.sciencedirect.com/science/article/pii/S0166218X25005761>.
- [16] Luca Mariot, Enrico Formenti, and Alberto Leporati. Constructing orthogonal latin squares from linear cellular automata. *arXiv*, 2016. URL <https://www.semanticscholar.org/paper/1abe43406189b39ff3064f2882467c2ae2a549c8>.
- [17] Luca Mariot, Martina Saletta, Alberto Leporati, and Luca Manzoni. Heuristic search of (semi-)bent functions based on cellular automata. *Natural Computing*, 2021. doi: 10.1007/s11047-022-09885-3. URL <https://www.semanticscholar.org/paper/d0ef2d96e201af37ae7bf3535d21a4ac29c6f6b3>.
- [18] A. B. Suksmono. Finding a hadamard matrix by simulated quantum annealing. *Entropy*, 2018. doi: 10.3390/e20020141. URL <https://www.semanticscholar.org/paper/d98246df005a52ff9a5e99b6c5f548c1def1ba12>.
- [19] A. B. Suksmono. A quantum approximate optimization method for finding hadamard matrices. *Scientific Reports*, 2025. doi: 10.1038/s41598-025-18778-1. URL <https://www.nature.com/articles/s41598-025-18778-1>.

- [20] A. B. Suksmono and Yuichiro Minato. Finding hadamard matrices by a quantum annealing machine. *Scientific Reports*, 2019. doi: 10.1038/s41598-019-50473-w. URL <https://www.semanticscholar.org/paper/c42e6a8a231cd8cd638ff953b6aa7b539c4596e9>.
- [21] A. B. Suksmono and Yuichiro Minato. Quantum computing formulation of some classical hadamard matrix searching methods and its implementation on a quantum computer. *Scientific Reports*, 2022. doi: 10.1038/s41598-021-03586-0. URL <https://www.semanticscholar.org/paper/1fd50d1147b7684686e533886b44a9e92ac2e807>.
- [22] Andri B. Suksmono. Finding a hadamard matrix by simulated annealing of spin vectors. *Journal of Physics: Conference Series*, 2016. doi: 10.1088/1742-6596/856/1/012012. URL <https://www.semanticscholar.org/paper/b01723f49ef6134f64d4675cc9a0d747eb29350b>.
- [23] Michail-Antisthenis I. Tsompanas, Nikolaos I. Dourvas, K. Ioannidis, G. Sirakoulis, R. Hoffmann, and A. Adamatzky. Cellular automata applications in shortest path problem. *Lecture Notes in Computer Science*, 2017. doi: 10.1007/978-3-319-77510-4_8. URL <https://www.semanticscholar.org/paper/4b85674750c1ad93a8d26427e52e7bec82d7e392>.